

BSc in IT: Specialising in Data Science

IT3021: Data Warehousing
and Business Intelligence

Lecture 03

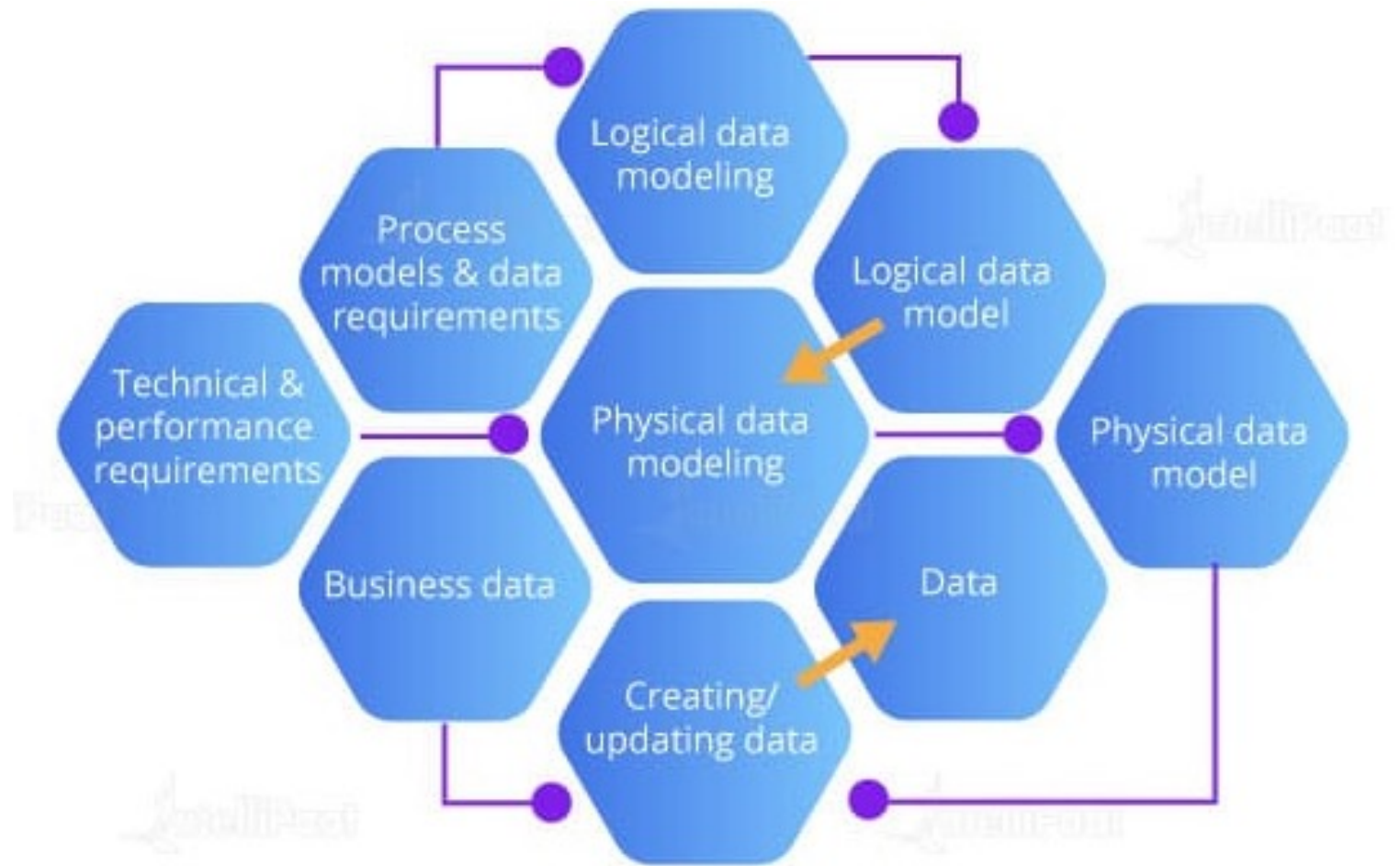
Dimensional Modelling

Part 01

Content

- Data modelling
- Dimensional modelling
 - Dimension tables
 - Facts and fact tables
 - Schemas
 - Star
 - Snowflake
 - Galaxy
 - Surrogate keys
- Dimensional modelling steps

Data Modelling



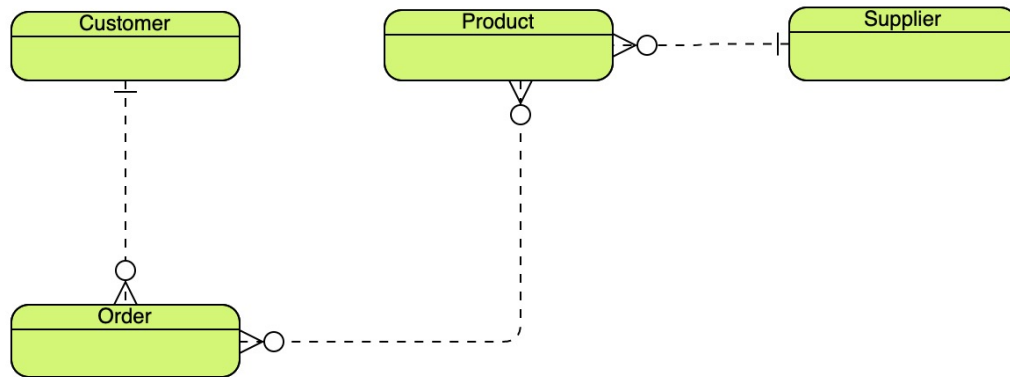
Data Modelling: What is it?

- Data modelling is the process of creating a data model for the data to be stored in a database
- It helps to visually represent data and enforces business rules, regulatory compliances, and government policies on the data
- It is like architect's building plan
- Emphasizes on what data is needed and how it should be organized instead of what operations need to be performed on the data
- Ensure consistency in naming conventions, default values, semantics, security, while ensuring quality of the data

Data Modelling: Why?

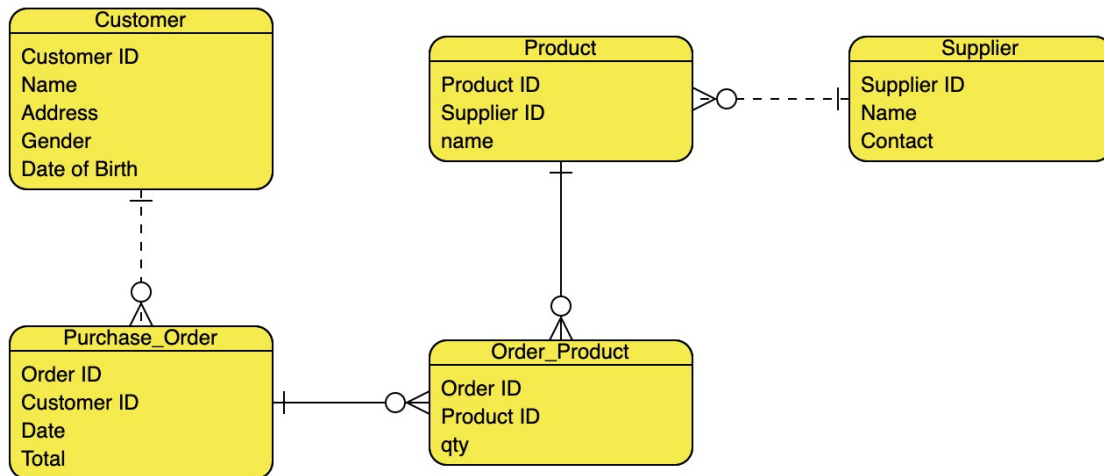
- All required data objects are accurately represented
- It provides a clear picture of the base data and can be used by database developers to create physical objects in a database
 - Start with a conceptual diagram, then a logical diagram, followed by a physical diagram
 - Finally provides you with relational tables, primary and foreign keys and stored procedures
- It is also helpful to identify missing and redundant data
- Though the initial creation of data model is labour and time consuming, in the long run, it makes your IT infrastructure upgrade and maintenance cheaper and faster.

Conceptual Model



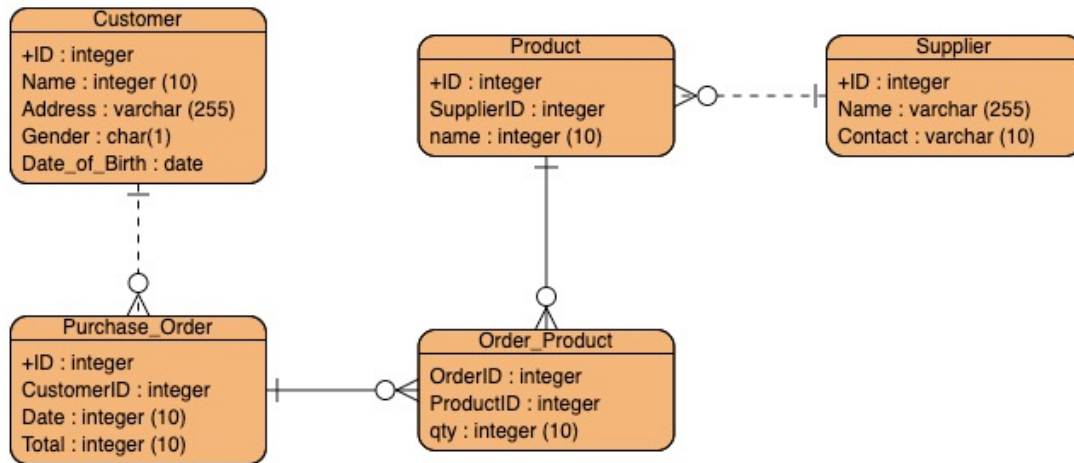
- Models the business objects that should exist in a system and the relationships between them
- A conceptual model is developed to present an overall picture of the system by recognizing the business objects involved
- It defines what entities and relationships exist, NOT tables
- Highly abstract

Logical Model



- Detailed version of a conceptual ERD: more of a relational model
- Attributes (including PKs and FKs) in each entity are defined and operational and transactional entities are introduced
 - User friendly attribute names
- Normalized and entity relationships are mapped into additional relations as required (e.g., M:N)
- Although a logical data model is still independent of the actual database system in which the database will be created, you can still consider that it affects the design

Physical Model



- Developed for a specific version of a DBMS, location, data storage or technology to be used in the project
 - Follow DBMS compatible names and data types
- Columns should have exact data types, lengths assigned and default values
- Constraints are defined

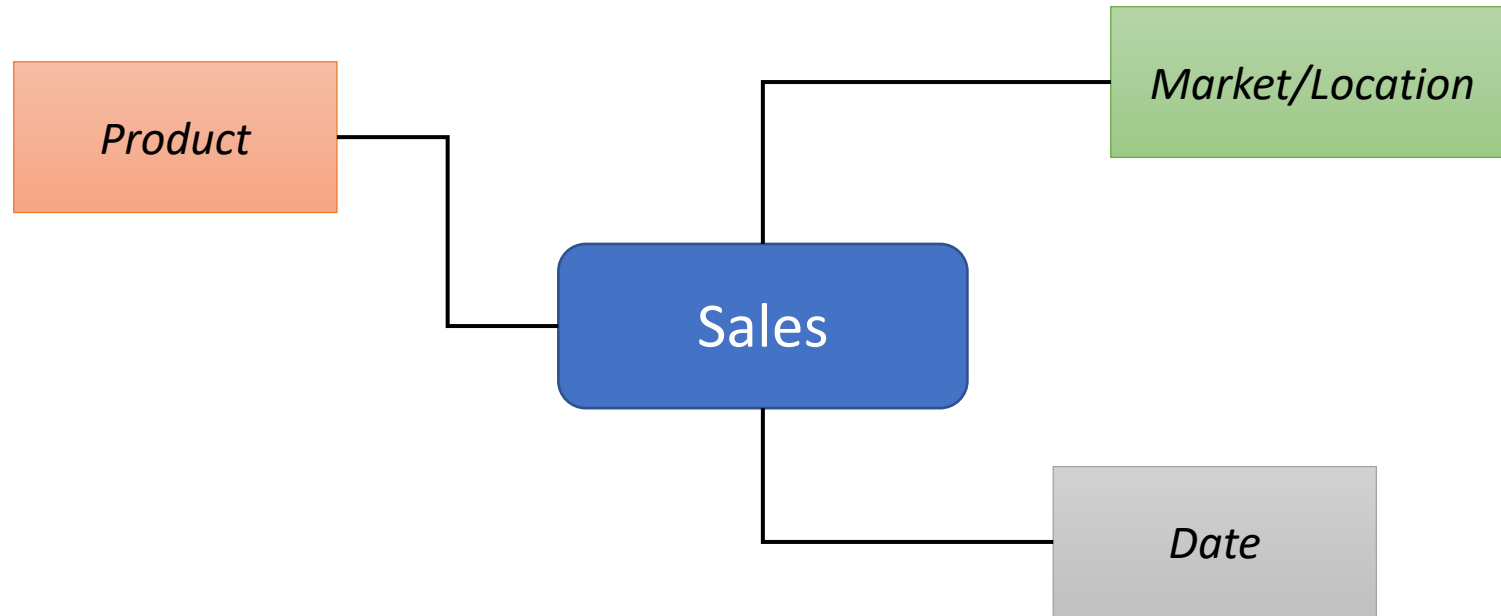
Dimensional Modelling



Dimensional Data Models

- The concept of Dimensional Modelling was developed by Ralph Kimball and is comprised of “*fact*” and “*dimension*” tables.
- A Dimensional model is designed to read, summarize, analyse numeric information like values, balances, counts, weights, etc. in a data warehouse.
- In contrast,
 - relational models are optimized for adding, updating and deleting of data in a real-time online transactional system (OLTP) and,
 - in the relational models, normalization reduce redundancy in data.
- On the contrary, dimensional model arranges data in such a way that it is easier to retrieve information and generate reports.

Dimensional Data Models for Analysis



*“We sell products in various markets (store locations). Thus, how to measure the performance of the organization in terms of sales, **product-wise, location-wise**, over a period of **time**?”*

Dimension Tables

- Dimensions provide the context surrounding a business process event, answering who, what, where, when, how & why like questions.
 - e.g., to understand sales performances, related dimensions would be:
 - *Who* – customer
 - *Where* – locations
 - *What* – products
- In other words, a dimension is a window to view measures/transactional values, providing meaning to analytical capabilities such as grouping, filtering, drilling down, etc.
- Dimensions encapsulates attributes (textual/descriptive context) associated with measures/values by separating and categorising these attributes into logical, distinct groups.
- Each dimension table is defined by a single primary key.
 - Surrogate Key: an integer value

Dimension Tables

Product Dimension
Product Key (PK)
SKU Number (Natural Key)
Product Description
Brand Name
Category Name
Department Name
Package Type
Package Size
Weight
Weight Unit of Measure
Storage Type
...

Attributes

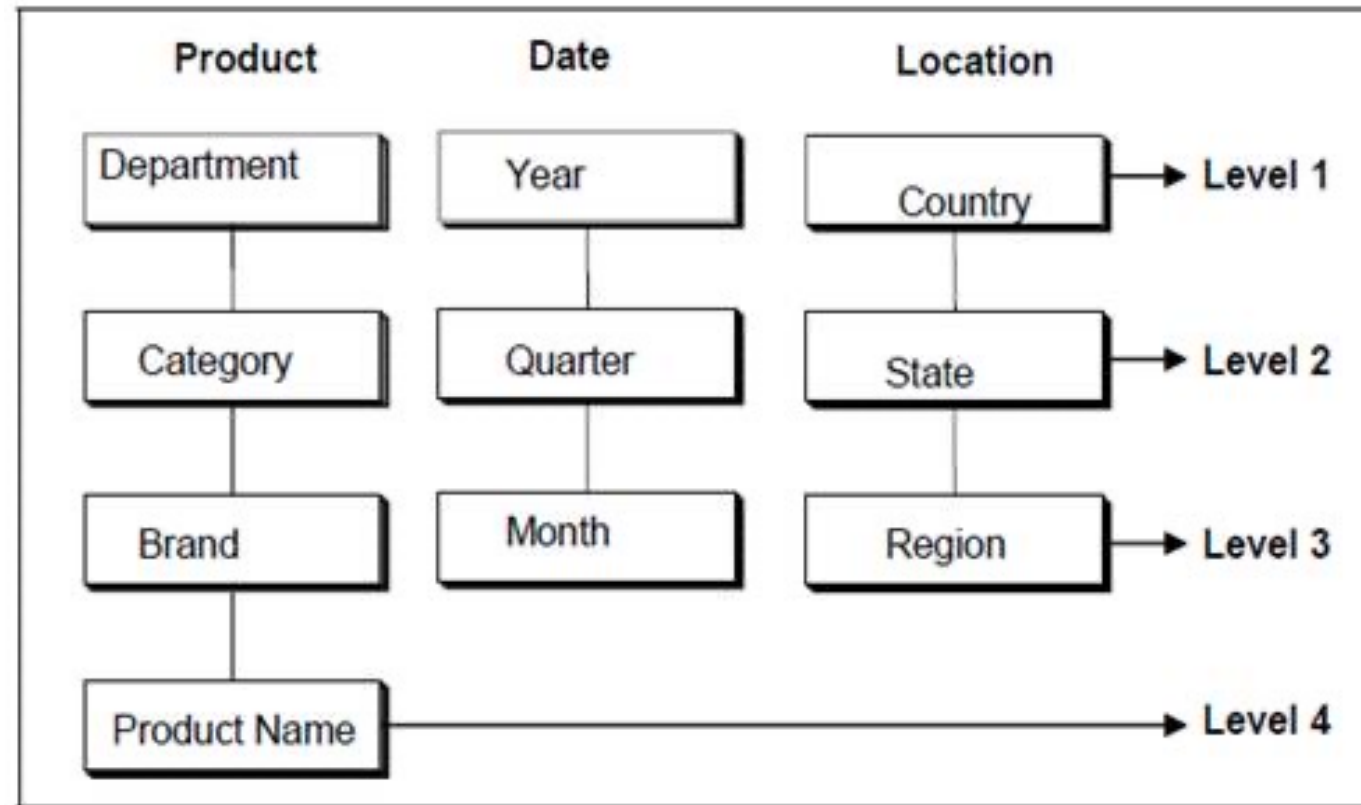
- Attributes are the various characteristics of the dimension
- In a “*Location*” dimension, attributes could be:
 - State
 - Country
 - Zip code
- Attributes are used to search, filter, or classify measures/transactional data
- Dimension tables contain *attributes* in columns

Hierarchies

- It is the specification of levels that represents relationships between different attributes within an entity (could be one or more dimensions)
- In other words, it is a series of parent-child relationships where parent members can be further aggregated as children of another parent
 - Hierarchies in *Date* dimension: **Year → Quarter → Month → Day**
 - Hierarchies in *Product* dimension: **Category → Brand → Product**
 - Hierarchies in *Region* dimension : **City → Region → Country**
- They provide a navigation path to drill down (deeper analysis)

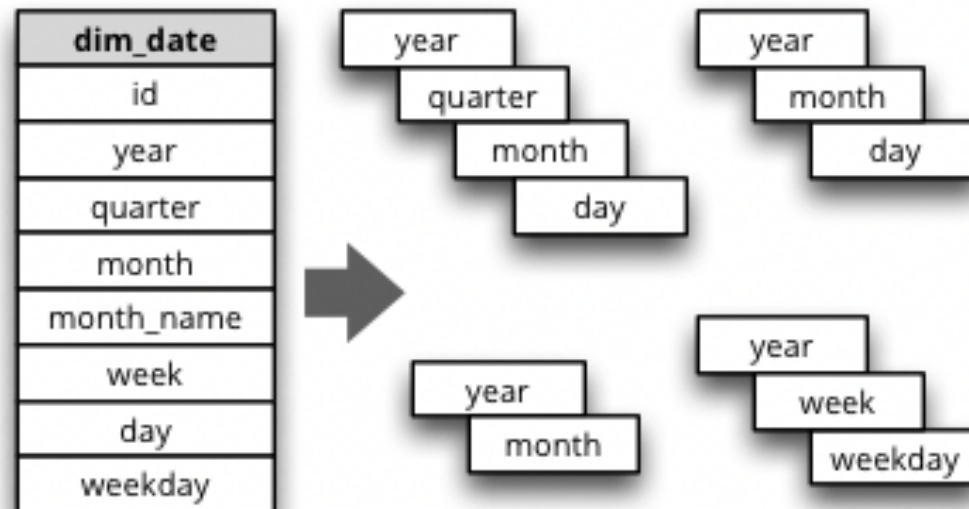
Hierarchies organize dimension data into levels, allowing users to drill down for detailed analysis.

Hierarchies



Hierarchies

- It is possible for a dimension to contain more than one hierarchy



Hierarchies

- What hierarchies can we derive?

ProductKey	ProductAlternateKey	ProductName	Color	ProductSubcategoryName	ProductCategoryName	ProductLine
222	HL-U509-B	Sport-100 Helmet, Blue	Blue	Helmets	Accessories	S
487	HY-1023-70	Hydration Pack - 70 oz.	Silver	Hydration Packs	Accessories	S
528	TT-M928	Mountain Tire Tube	NA	Tires and Tubes	Accessories	M
529	TT-R982	Road Tire Tube	NA	Tires and Tubes	Accessories	R
341	BK-R50B-48	Road-650 Black, 48	Black	Road Bikes	Bikes	R
345	BK-M82S-42	Mountain-100 Silver, 42	Silver	Mountain Bikes	Bikes	M
563	BK-T79Y-54	Touring-1000 Yellow, 54	Yellow	Touring Bikes	Bikes	T
237	LJ-0192-X	Long-Sleeve Logo Jersey, XL	Multi	Jerseys	Clothing	S
445	SH-M897-S	Men's Sports Shorts, S	Black	Shorts	Clothing	S
463	GL-H102-S	Half-Finger Gloves, S	Black	Gloves	Clothing	S
424	RW-R820	HL Road Rear Wheel	Black	Wheels	Components	R
427	FR-M63B-44	ML Mountain Frame - Black, 44	Black	Mountain Frames	Components	M
430	FR-R72Y-40	ML Road Frame-W - Yellow, 40	Yellow	Road Frames	Components	R

Facts and Fact Tables

Facts/Measures

- Facts are the measurements/metrics/transaction values generated from the business process
 - e.g., sales numbers

Fact Tables

- Stores,
 - the measures of interest (facts) and,
 - foreign keys to dimension tables.
- These measures are stored in the fact table with the appropriate granularity, or level of detail: **“Grain”**
 - e.g., one record in a *Sales* fact table per line item sold in each transaction
- Fact table primary key is a composite key made up of all or a subset of foreign keys

Grain means the level of detail in the fact table.

Eg : In a Sales fact table, one row may represent each line item sold per transaction.

Fact Tables

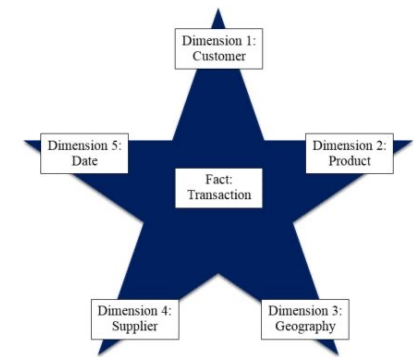
Retail Sales Facts
Date Key (FK)
Product Key (FK)
Store Key (FK)
Promotion Key (FK)
Customer Key (FK)
Clerk Key (FK)
Transaction #
Sales Dollars
Sales Units

Schemas for Dimensional Modelling

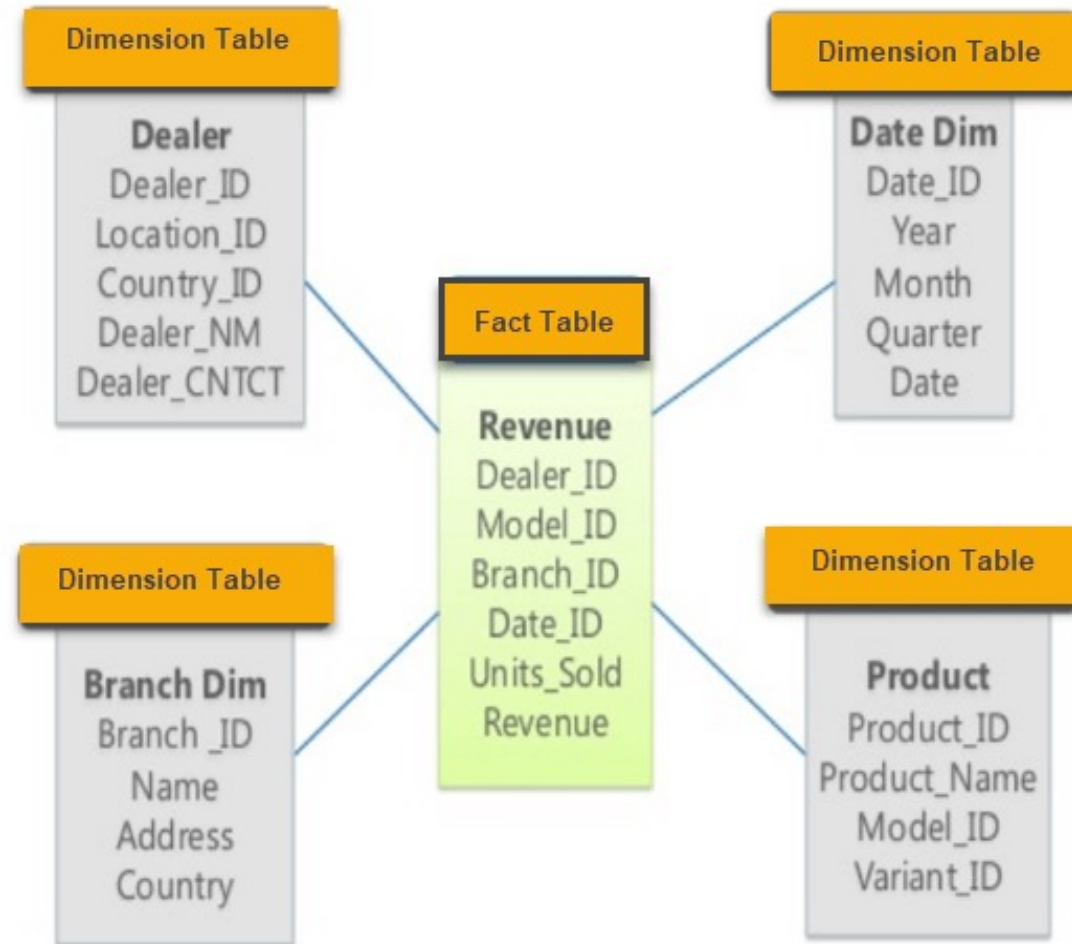
- Fact tables and dimension tables are joined to define a dimensional model
- Schemas provide techniques to join facts and dimensions
- Types of schemas:
 - Star
 - Snowflake
 - Galaxy

Star Schema

- Single object (fact table) sits in the middle and connected to other surrounding objects (dimension tables) like a star
- A simple star consists of one fact table; a complex star can have more than one fact table
- Each and every dimensional entity in a star schema is represented with only one dimension table
- Dimension tables are not normalized
- Dimension table is joined to the fact table
- Dimension table are not joined to each other



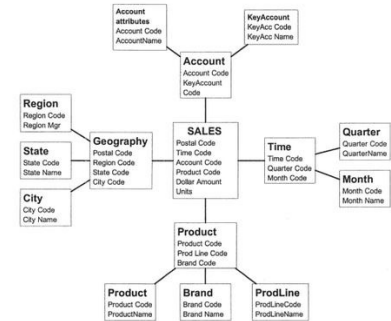
Star Schema



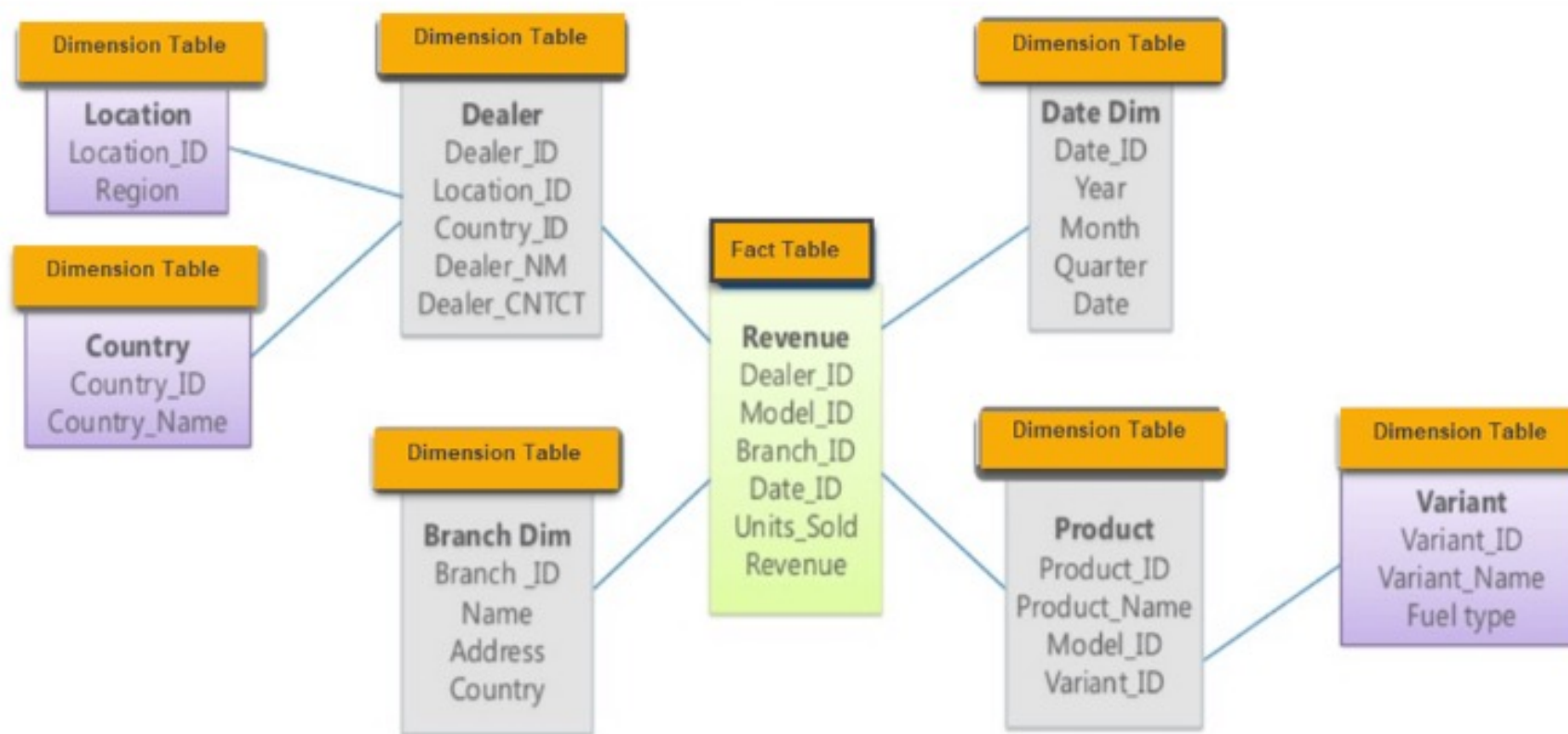
Example of Star Schema

Snowflake Schema

- An extension of the star schema, where one or more surrounding point of the star expands into more points
- Some dimension tables in the snowflake schema are **normalized**, each representing one or more levels in the dimensional hierarchy
- The main benefit of the snowflake schema is it uses smaller disk space
- Easier to expand a dimension: add a new table to the schema
- Due to multiple tables, query performance is reduced
- More maintenance efforts, because of the more lookup tables



Snowflake Schema



Star vs. Snowflake

- By design:

Star	Snowflake
Contains a fact table surrounded by dimension tables	Contains a fact table surrounded by dimension tables, which are in turn surrounded by more dimension tables
A single join with the fact table creates the relationship between the fact and the dimensional entity	Multiple joins are required to create the relationship between the fact and the dimensional entity
Hierarchies for the dimension are stored in one dimension table	Hierarchies for the dimension are divided into separate table
Simple DB design	Complex DB design
Denormalized data structures	Normalized data structures
High level of data redundancy	Low level of data redundancy

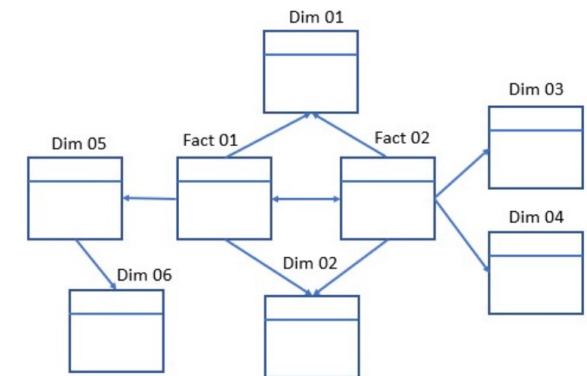
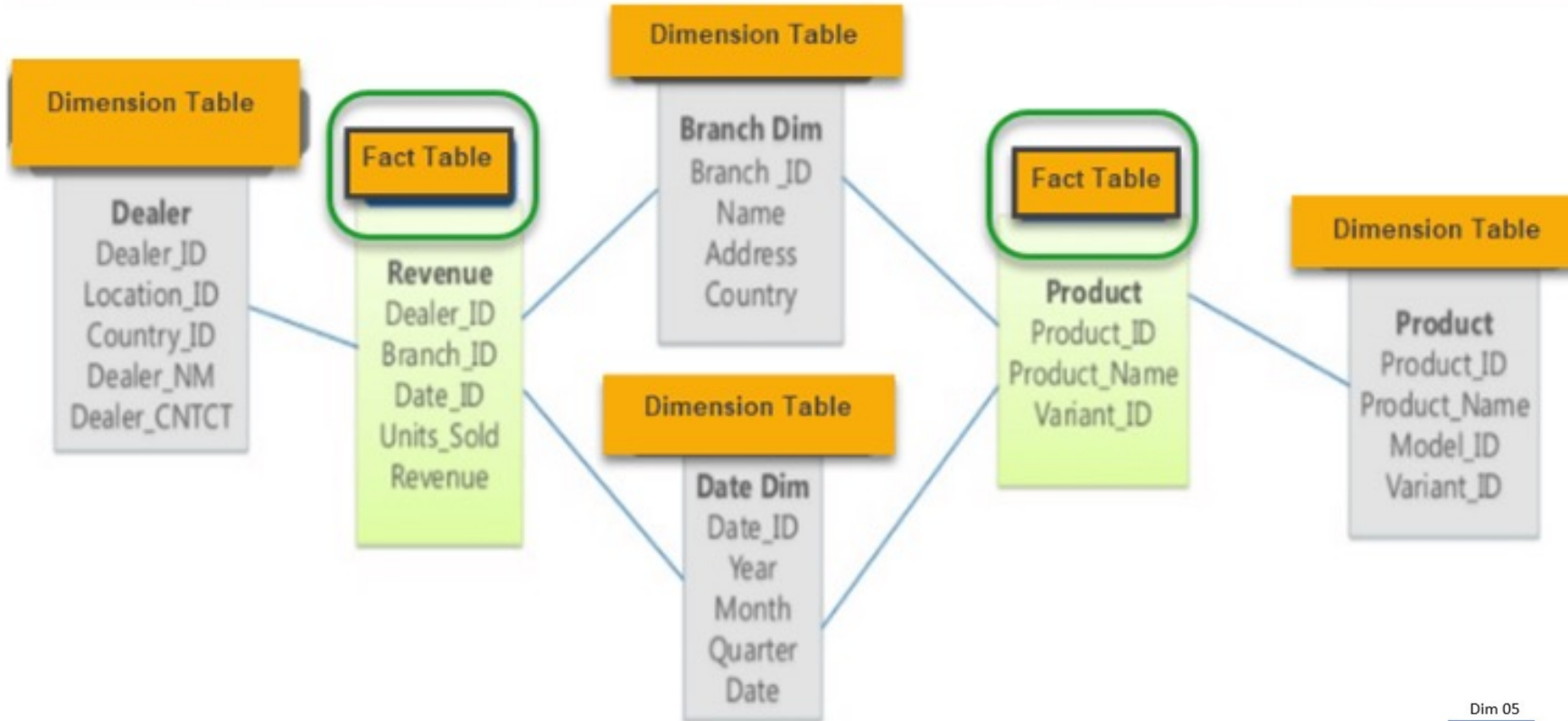
Star vs. Snowflake

- Model simplicity (Star: one-dimension table to work with)
- Storage efficiency (Snowflake: no redundancy)
- ETL complexity (Star: less lookups)
- Query performance:
 - SQL table joins (Star) vs. I/O performance (Snowflake)
 - In general, the simplified joins of a star design give the best performance for smaller tables (few columns and thousands of rows in dimension), whereas the snowflake design has better performance for larger tables (many columns and millions of rows in dimension)
- Analytical requirements:
 - Metrics analysis: “what is the revenue for a given customer” (Star)
 - Dimension analysis: “how many customers from a given region” (Snowflake)

Galaxy Schema

- A galaxy schema contains two or more fact tables that share dimension tables
- Galaxy schema can be derived as a collection of star schemas interlinked and completely normalized, to avoid redundancy
 - Since it is viewed as a collection of star schemas, it is also called as a Fact Constellation Schema (same reason to be called as galaxy too)
- It can also be a meaningful association of a Snowflake schema with a Star schema, where the fact tables of both schemas can be linked
- The dimensions in this schema are normalized into separate dimensions based on the various levels of hierarchy
 - e.g., If geography has four levels of hierarchy like region, country, state, and city then galaxy schema should have four dimensions
- Moreover, it is possible to build this type of schema by splitting the one-star schema into more Star schemes

Galaxy Schema



Database Keys

- **Key**: is one or more data attributes that uniquely identify an occurrence in an entity. In a physical database, a key would be formed of one or more table columns whose value(s) uniquely identifies a row within a relational table
- **Natural key**: A key that is formed of attributes that already exist in the real world
 - e.g., NIC, PP Number
- **Candidate key**: Minimal set of attributes that uniquely identifies each occurrence of an entity
- **Primary key**: Candidate key selected to uniquely identify each occurrence of an entity
- **Composite key**: A candidate key that consists of two or more attributes
- **Foreign key**: One or more attributes in an entity that represents a key, either primary or secondary, in another entity
- **Alternate key**: Also known as a secondary key, is another unique identifier from the remaining candidate keys, when the primary key is selected
- **Surrogate key**: A key with no business meaning

Surrogate Keys

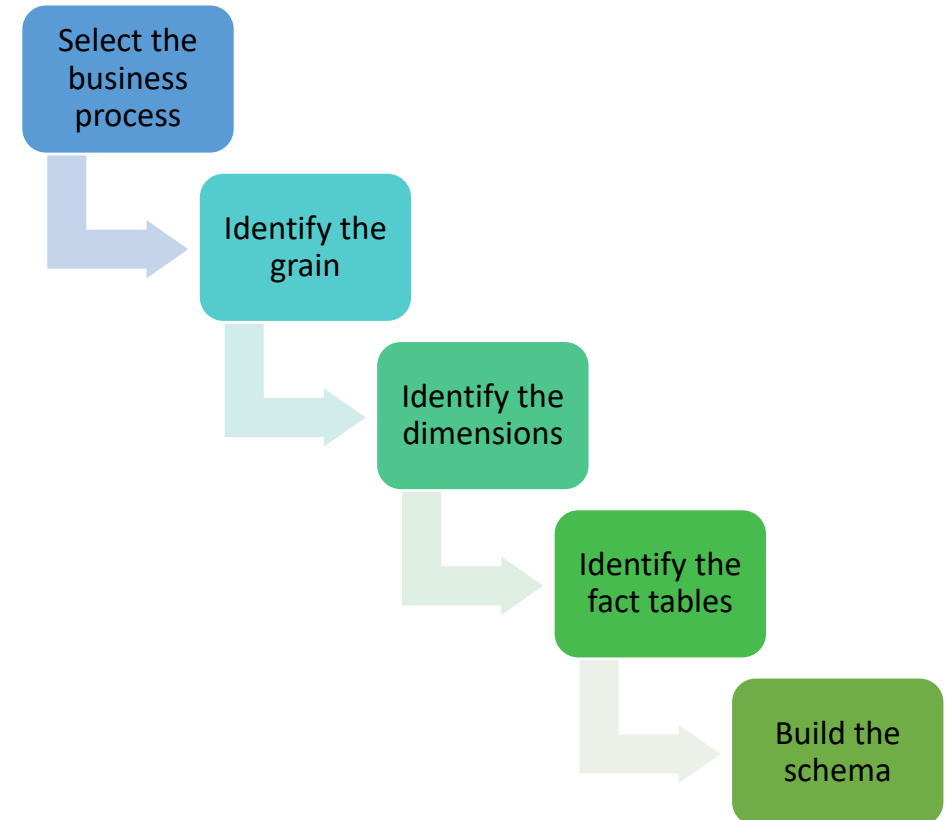
- A unique primary key for dimension tables
- Cannot be the operational system's natural key
 - Duplicates when track changes over time
 - Created by more than one source system
 - Performance issues – text fields
- Simple integers, assigned in sequence
- The date dimension is a special case

Surrogate Keys: Advantages

- Protects the DW system from unexpected administrative changes in sources
- Allow the system to integrate the same data
- Provide the means for tracking changes in dimension attributes over time
- Integer surrogate keys can improve query and processing performance compared to larger character (text) keys

Steps of Dimensional Modelling

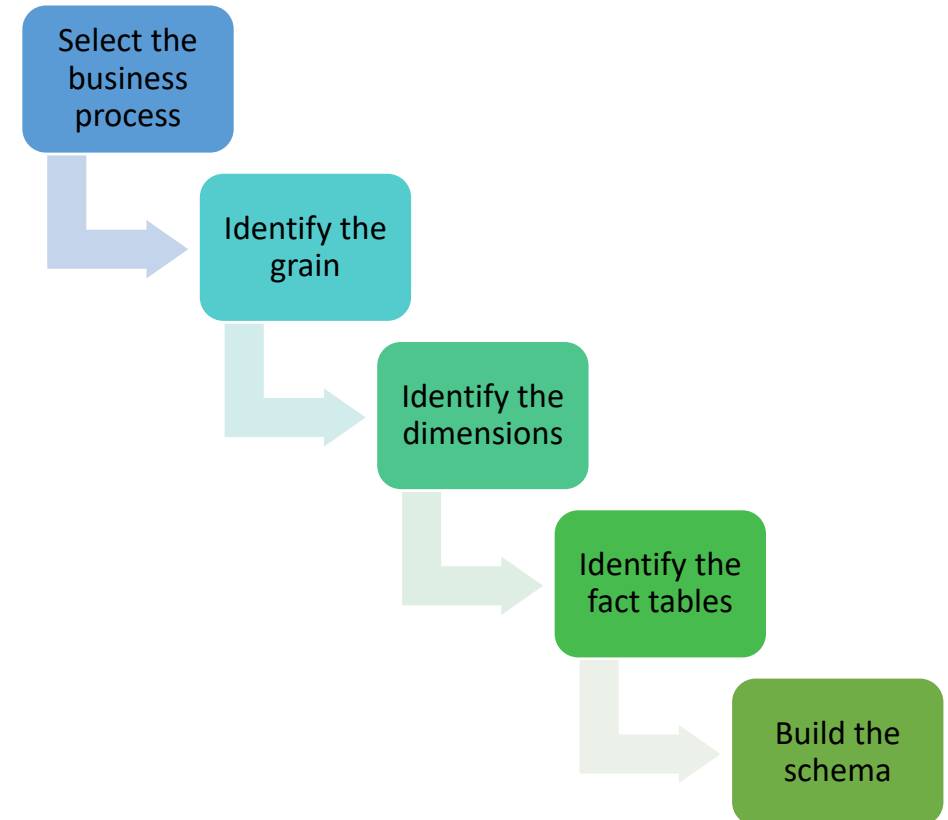
- Step 1 - Identify the business process:
 - Identifying the actual business process the data warehousing solution should address
 - This could be marketing, sales, hr, etc. as per the analytical needs of the organization
 - The selection of the business process also depends on the availability and the quality of data for that process
 - A failure in this process would impact in cascading and irreparable defects
 - To describe the business process, plain text or basic Business Process Modelling Notation (BPMN) or Unified Modelling Language (UML) can be used



Steps of Dimensional Modelling

- Step 2 - Identify the grain:

- It is the process of identifying the lowest level of information for tables (including aggregate tables) in your data warehouse model
 - e.g., Sales data – line item wise for each order
- Store aggregate data on a monthly, weekly, daily or hourly basis, based on the nature of reports and frequent analysis
 - e.g., The CEO wants to find the sales for specific products in different locations on a daily basis
- In general, data warehouse fact tables store data in the lowest possible granular level



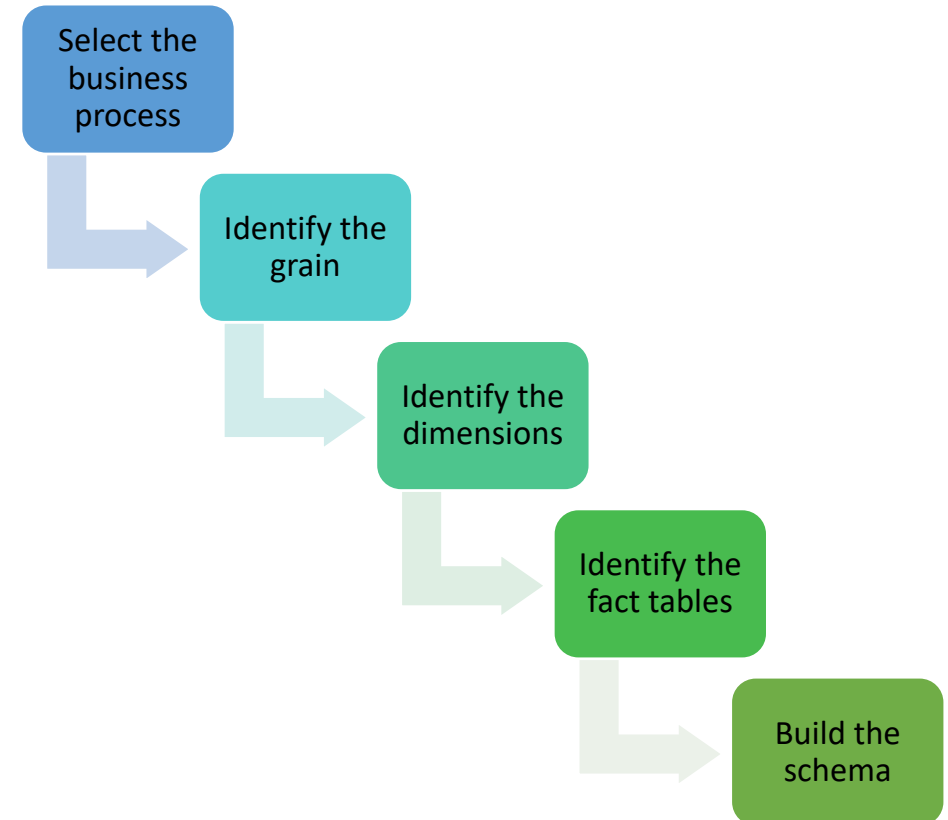
Steps of Dimensional Modelling

- Step 3 - Identify the dimensions:

- Dimensions are nouns like date, store, inventory, etc.
- Dimensions are where all the descriptive (non-measurable data) should be stored
 - For example, the *Date* dimension may contain data like a year, month and weekday

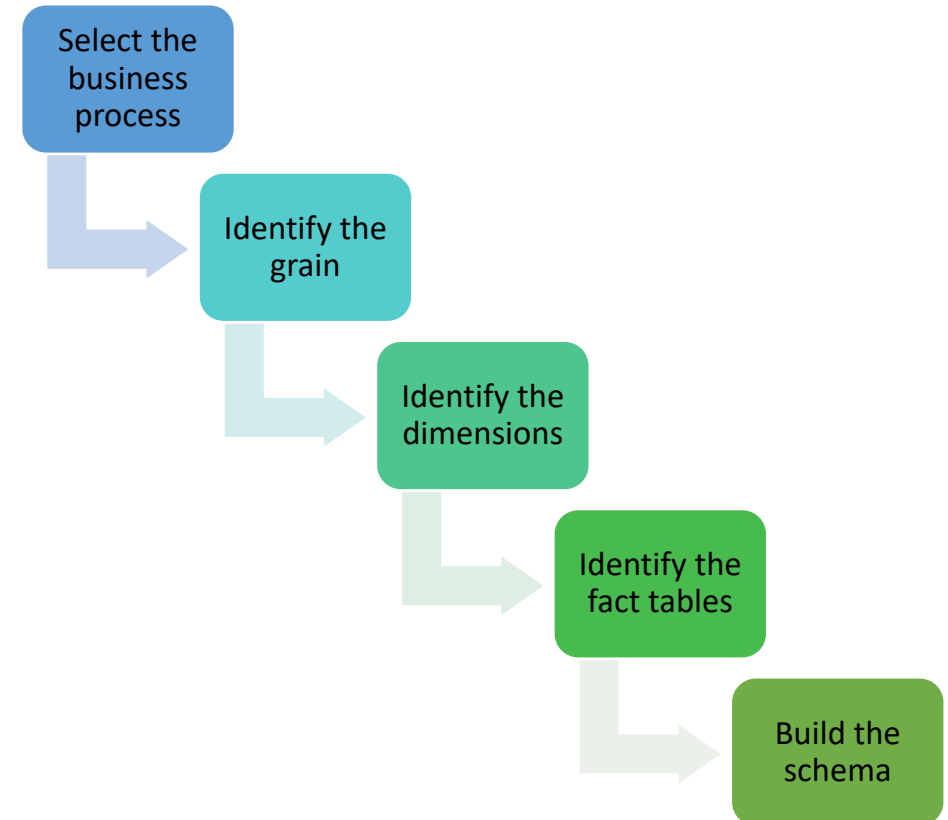
- Example of Dimensions:

- Dimensions: Product, Location and Time
- Attributes: for *Product*: Product key (SK), SKU, Name, Type, Specifications
- Hierarchies: for *Location*: Country, State, City, Street Address, Name



Steps of Dimensional Modelling

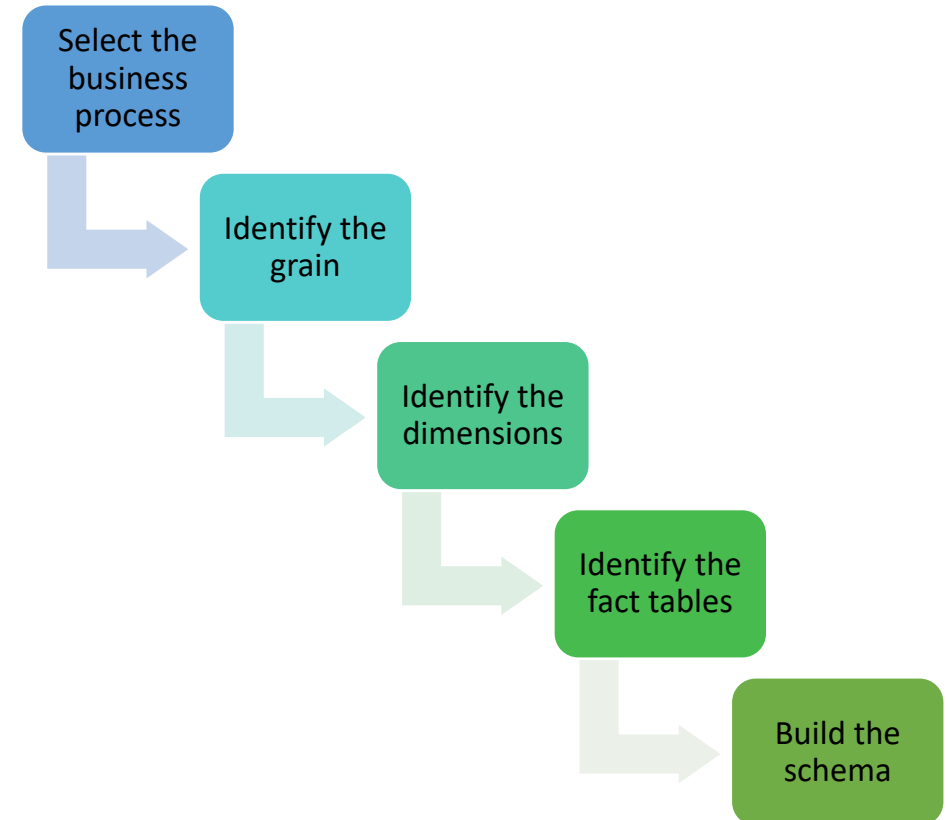
- Step 4 - Identify fact tables:
 - This step is co-associated with the business users of the system because this is where they get access to data stored in the data warehouse
 - Most of the fact table rows are numerical values like price or cost per unit, etc.
 - Example of Facts:
 - Sales
 - Deliveries
 - Payments



Steps of Dimensional Modelling

- Step 5 - Build the Schema:

- A schema is nothing but the database structure (arrangement of tables)
- Identify the schema required as per the business/analytical requirements
- Design the schema:
 - Star Schema
 - Snowflake Schema



General Rules for Dimensional Modelling

- Load atomic data into dimensional structures
- Build dimensional models around business processes
- Need to ensure that every fact table has an associated date dimension table
- Ensure that all facts in a single fact table are at the same grain or level of detail
- It's essential to store report labels and filter domain values in dimension tables
- Need to ensure that dimension tables use a surrogate key
- Continuously balance requirements and realities to deliver business solution to support their decision-making